Implementing Support Vector Machines (SVM) from scratch

Objective

The goal is to implement the **Support Vector Machine (SVM)** classification algorithm (from scratch or minimally assisted), understand the concept of margin maximization, and analyze the impact of kernel choice and hyperparameters.

Dataset

Use the **Breast Cancer Wisconsin Dataset**:
Breast Cancer Wisconsin Dataset
Link: https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic)

- Binary classification problem (Malignant vs Benign)
- 30 numerical features

Part (a): Implementation of the SVM Model [50]

1. Understanding the SVM Algorithm:

Support Vector Machine (SVM) is a supervised learning algorithm used for classification. It is a **margin-based classifier** that aims to find the optimal decision boundary between classes. To classify data, it follows these steps:

1. **Initialize Parameters:**
   Initialize the weight vector (w) and bias (b), which define the decision boundary (hyperplane).
2. **Compute Decision Function:**
   For each training point, compute the value of the decision function:

$$f(x) = w \cdot x + b$$

3. **Apply Hinge Loss Condition:**
   Check whether each data point satisfies the margin condition:

$$y_i(w \cdot x_i + b) \geq 1$$

   - If the condition is satisfied, the point is correctly classified and lies outside the margin.

- If not, it contributes to the loss and requires updating.

4. **Update Parameters:**
   Adjust the weights and bias using gradient descent:
   - Apply only regularization if correctly classified
   - Apply both classification loss and regularization if misclassified
5. **Repeat Optimization:**
   Iterate over the dataset multiple times to minimize the objective function and find the optimal hyperplane.
6. **Make Predictions:**
   For a new data point, compute:

$$f(x) = w \cdot x + b$$

   - If $f(x) \geq 0$, assign one class
   - Otherwise, assign the other class

2. Dataset Preparation

- Use all features and both classes
- Convert labels to binary format (e.g., 0 and 1 or -1 and +1)

3. Data Preprocessing

- Standardize features (mean = 0, variance = 1)
- Important because SVM is sensitive to feature scaling

4. Task: Implement the SVM Model

Implement a class `SupportVectorMachine` with following methods:

Required methods:

- `__init__(learning_rate, lambda_param, n_iters)` : To intialize the SVM with specified hyperparameters.
- `fit(X_train, y_train)` : To train the model using the training data.
- `predict(X_test)` : To make predictions on new data.

Implementation Guidelines

You may implement a **linear SVM using gradient descent**:

- Objective function (hinge loss with regularization):
  Minimize:

$$\frac{1}{2}||w||^2 + \lambda \sum \max(0, 1 - y_i(w \cdot x_i + b))$$

- Steps:
  i. Initialize weights and bias
  ii. Iterate over dataset
  iii. Update weights based on hinge loss condition
  iv. Repeat for specified number of iterations

5. Training the Model

Split the preprocessed Breast Cancer Wisconsin dataset into a training set (80%) and a testing set (20%). Use the fit method to prepare your model.

Part (b): Evaluation and Visualization [50]

1. Hyperparameter Tuning

Evaluate performance for different values of:

- Regularization parameter: λ (e.g., 0.001, 0.01, 0.1, 1)
- Learning rate

Report:

- Accuracy on test set
- Present results in a table

2. Kernel Exploration

Train models using:

- Linear kernel
- Polynomial kernel
- RBF (Gaussian) kernel

(You may use **sklearn ONLY for kernel comparison**, not for core implementation.)

Compare:

- Accuracy
- Decision boundary complexity

3. Visualization

Since the dataset has many features:

Step 1: Reduce to 2 features

Choose any two features (e.g., mean radius, mean texture)

Step 2: Plot decision boundaries

Create plots for:

- Linear SVM
- RBF kernel SVM

Explain:

- Linear separability vs non-linear boundaries

4. Analysis and Report

Write a brief but well-structured report covering the following aspects of your work:

Implementation

- Describe how you implemented the SVM algorithm from scratch.
- Explain the key steps involved, including parameter initialization, optimization, and prediction.
- Highlight any challenges you faced during implementation (e.g., handling convergence, tuning parameters, numerical stability) and how you addressed them.

Hyperparameter Analysis

- Analyze the effect of the regularization parameter (λ) on model performance.
- Discuss how different values of λ influence the decision boundary.
- Explain the concepts of **underfitting** and **overfitting** in the context of SVM, and relate them to your experimental results.

Kernel Comparison

- Compare the performance of different kernels (e.g., linear, polynomial, RBF).

- Discuss scenarios where the linear kernel fails to capture complex patterns in the data.
- Explain why the RBF (Gaussian) kernel is more effective for non-linear and complex datasets.

Conceptual Insight

- Explain the concept of **margin maximization** and why it is important in SVM.
- Describe the role of **support vectors** and how they influence the final decision boundary.

Deliverables

Submit the following components as part of your assignment:

- Source code for your SVM implementation
- A table showing accuracy results for different hyperparameter settings
- Relevant plots, including:
  - Decision boundary visualizations
  - Performance comparison graphs
- A final report (in PDF format) summarizing your methodology, results, and analysis

Bonus Challenge

1. **Soft Margin vs Hard Margin**
   - Experiment with different λ values
   - Observe effect on margin violations
2. **Compare with k-NN**
   - Compare performance with your previous k-NN implementation
   - Discuss:
     - Speed
     - Accuracy
     - Interpretability
3. **Use PCA for Visualization**
   - Reduce dataset to 2D using PCA
   - Plot decision boundary in reduced space

Important Note

- Do NOT use:
  - `sklearn.svm.SVC` for core implementation (Part a)
- You MAY use sklearn:
  - For comparison (Part b only)